MODULE 35 · DOMAIN 4 - RESILIENCE, MIGRATION AND CONTINUITY

Replatforming and Refactoring

Swap self-managed parts for managed services with little code change, and break monoliths apart one route at a time with the strangler fig pattern.

DOMAIN 4 of 5 EXAM WEIGHT 15% MODULE 35 of 52

admin@sthithapragna.com

Replatform or refactor?

REPLATFORM

Lift, tinker and shift

- Managed database, broker, container platform
- Little or no code change
- Removes undifferentiated operations
- Weeks per application

REFACTOR

Change the architecture

- Microservices, events, serverless
- Significant code change
- Unlocks scale and release speed
- Months, done incrementally

THE TIEBREAKER

Reduce operations with little code change: replatform. New capabilities the old design blocks: refactor.

Common replatform moves

FROM	TO	REMOVES
Self-managed MySQL, PostgreSQL	RDS or Aurora	Patching, backups, failover
Oracle or SQL Server on servers	RDS for that engine	Infrastructure work, keeps engine
App servers on VMs	ECS on Fargate	Server management
Scheduled scripts on a server	Lambda with EventBridge Scheduler	An always-on server
ActiveMQ or RabbitMQ	Amazon MQ	Broker operations, same protocols
Self-managed Kafka	Amazon MSK	Cluster operations

ON THE EXAM

Keep the protocol or engine the code expects; change only who runs it.

Big-bang rewrites fail. Replace a monolith **one capability at a time**, while it keeps serving customers.

The strangler fig pattern

01
Front door
Put a router in front of the monolith

02
Build
Implement one capability as a new service

03
Route
Send that path to the new service

04
Retire
Remove the code from the monolith

05
Repeat
Until the monolith is gone

ON THE EXAM

The router is the key piece: ALB rules, API Gateway routes or CloudFront behaviours.

Patterns that make decomposition work

- **Anti-corruption layer** translates between old and new models.
- **Database per service**, fed by events or CDC, not a shared schema.
- **Events** through EventBridge or SNS decouple services.
- **Branch by abstraction** swaps implementations behind an interface.
- **Sagas** replace distributed transactions across services.

> ON THE EXAM
>
> New services reading the monolith's tables directly recreate the coupling you are removing.

Modernisation tooling, now

NEED	CURRENT OPTION
Incremental routing for strangler fig	ALB, API Gateway, CloudFront routing
Managed strangler fig environment	Refactor Spaces: closed to new customers
.NET Framework on Windows to Linux	AWS Transform for .NET
Assess and plan modernisation paths	AWS Transform
Containerise applications	AWS Transform, then ECS or EKS

ON THE EXAM

Migration Hub Refactor Spaces closed to new customers on 7 November 2025, with Migration Hub.

Reading the scenario

WHEN THE QUESTION SAYS	IT IS TESTING
Reduce DBA work, no code change	Replatform to RDS or Aurora
Containers, no servers to manage	ECS on Fargate
Replace the monolith gradually	Strangler fig behind a router
New service needs old data, no coupling	Anti-corruption layer, events
.NET on Windows, cut licence cost	AWS Transform for .NET
Self-managed broker, same protocols	Amazon MQ

ON THE EXAM

Prefer the least change that meets every requirement, and change incrementally when more is needed.

> Meridian Freight runs a self-managed MySQL 8.0 cluster on EC2 for its booking system. Two DBAs spend most of their time on patching, backups and replica failover.
>
> The application must not be rewritten.

What should the solutions architect recommend?

A Refactor the data layer to DynamoDB

B Automate patching on EC2 with Systems Manager

C Rehost the cluster with AWS Transform MGN

D Replatform to Amazon Aurora MySQL

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A DynamoDB

Requires rewriting the application, which is ruled out.

B Automate patching

Leaves backups, failover and the rest self-managed.

C Rehost

Already on EC2; moves nothing off the DBAs.

D Aurora MySQL

Managed patching, backups and failover; MySQL-compatible.

THE SIGNAL

Operational pain, same engine, no rewrite: replatform to a managed database.

> Meridian Freight's Java tracking application runs on 30 EC2 instances that the operations team patches monthly. The team has already containerised it for testing.
>
> Operations wants to stop managing servers, with no changes to the application code.

What should the solutions architect recommend?

A Run the containers on Amazon ECS with AWS Fargate

B Amazon EKS with self-managed node groups

C Rewrite the application as Lambda functions

D Larger EC2 instances, so there are fewer to patch

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A ECS on Fargate

Runs the existing containers with no servers to manage.

B EKS self-managed nodes

Still servers to patch, plus Kubernetes to run.

C Lambda rewrite

A code change, which is ruled out.

D Fewer, larger instances

Still patching servers.

THE SIGNAL

Containers exist, no servers wanted, no code change: ECS on Fargate.

> Meridian Freight wants to replace its customer portal monolith gradually. The tracking pages should move first to a new service, while every other page keeps running on the monolith.
>
> Customers must not notice any change of URL, and the team wants to move one feature at a time.

What should the solutions architect implement?

A. Rewrite the whole portal, then switch DNS

B. Put an Application Load Balancer in front of the monolith, and add a path rule sending /tracking to the new service

C. Create AWS Migration Hub Refactor Spaces environments

D. Give the new service its own domain name

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Big-bang rewrite

The opposite of one feature at a time.

B ALB path routing

The strangler fig front door; move routes one by one.

C Refactor Spaces

Closed to new customers since November 2025.

D New domain

Customers would see the change.

THE SIGNAL

Gradual replacement behind the same URL: a router in front, one path at a time.

> Meridian Freight's new tracking service needs shipment data that the monolith stores in its own relational schema. The monolith's schema changes often and is poorly documented.
>
> The new service must not break when the monolith's schema changes.

How should the new service get the data?

A Query the monolith's database tables directly

B Copy the monolith's tables to the new service nightly

C Publish shipment events from the monolith, via change data capture, through an anti-corruption layer into the service's own data store

D Call the monolith's stored procedures

◆ PAUSE AND DECIDE

How should the new service get the data?

A Direct queries

Breaks whenever the monolith's schema changes.

B Nightly copy

Stale data, and still bound to the old schema.

C Events through an ACL

The translation absorbs schema changes; the service owns its data.

D Stored procedures

Couples the service to the monolith's internals.

THE SIGNAL

New services own their data; translate the old model at the boundary.

> Meridian Freight has 60 ASP.NET applications on .NET Framework running on Windows Server EC2 instances. Windows licensing is a large share of their cost.
>
> The team wants to modernise the portfolio consistently and run the applications on Linux.

What should the solutions architect recommend?

A Rehost the instances with AWS Transform MGN

B Buy Reserved Instances for the Windows servers

C Rewrite each application in Java by hand

D Use AWS Transform for .NET to port the applications to cross-platform .NET for Linux

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Rehost

Still Windows, still licensed.

B Reserved Instances

Discounts the licence cost; does not remove it.

C Manual Java rewrite

Slow, risky and inconsistent across 60 apps.

D AWS Transform for .NET

Agentic porting to Linux, consistent across the portfolio.

THE SIGNAL

.NET on Windows, move to Linux at portfolio scale: AWS Transform for .NET.

> Meridian Freight's depot systems publish messages to a self-managed ActiveMQ broker using JMS and AMQP. The broker crashes several times a year and nobody wants to maintain it.
>
> The depot systems cannot be changed this year.

What should the solutions architect recommend?

A Amazon MQ for ActiveMQ in an active/standby deployment

B Amazon SQS with code changes in each depot system

C Amazon MSK

D Amazon EventBridge

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Amazon MQ

Same protocols, managed and highly available.

B SQS

Requires code changes, which are ruled out.

C MSK

Kafka protocol, not JMS or AMQP.

D EventBridge

Different API; the depot systems would need changes.

THE SIGNAL

Existing JMS or AMQP clients that cannot change: Amazon MQ.

What to carry out of this module

- **Replatform**: same interface, managed operator, little code change.
- **Refactor incrementally**: strangler fig behind a router.
- **Own your data**: events, CDC and an anti-corruption layer.
- **AWS Transform for .NET** to move Windows .NET to Linux.
- **Refactor Spaces** is closed to new customers; routing rules do the job.

> ON THE EXAM
>
> Least change that meets the need, and incremental change when more is needed.